

Chapter 15. Files and Streams

Most of the techniques I've shown so far in this book revolve around the information that lives in objects and variables. This kind of state is stored in a particular process's memory, but to be useful, a program must interact with a broader world. This might happen through UI frameworks, but there's one particular abstraction that can be used for many kinds of interactions with the outside world: a *stream*.

Streams are so widely used in computing that you will no doubt already be familiar with them, and a .NET stream is much the same as in most other programming systems: it is simply a sequence of bytes. That makes a stream a useful abstraction for many commonly encountered features such as a file on disk or the body of an HTTP response. A console application uses streams to represent its input and output. If you run such a program interactively, the text that the user types at the keyboard becomes the program's input stream, and anything the program writes to its output stream appears on screen. A program doesn't necessarily know what kind of input or output it has, though—you can redirect these streams with console programs. For example, the input stream might actually provide the contents of a file on disk, or it could even be the output from some other program.

NOTE

Not all I/O APIs are stream-based. For example, in addition to the input stream, the `Console` class provides a `ReadKey` method that gives information about exactly which key was pressed, which works only if the input comes from the keyboard. So, although you can write programs that do not care whether their input comes interactively or from a file, some programs are pickier.

The stream APIs present you with raw byte data. However, it is possible to work at a different level. For example, there are text-oriented APIs that can wrap underlying streams, so you can work with characters or strings

instead of raw bytes. There are also various *serialization* mechanisms that enable you to convert .NET objects into a stream representation, which you can turn back into objects later, making it possible to save an object's state persistently or to send that state over the network. I'll show these higher-level APIs later, but first, let's look at the stream abstraction itself.

The Stream Class

The `Stream` class is defined in the `System.IO` namespace. It is an abstract base class, with concrete derived types such as `FileStream` or `GZipStream` representing particular kinds of streams. [Example 15-1](#) shows three of the `Stream` class's members. It has several other members, but these are at the heart of the abstraction. (As you'll see later, there are also asynchronous versions of `Read` and `Write`. The latest .NET versions also provide overloads that take one of the *span* types described in [Chapter 18](#) in place of an array. Everything I say in this section about these methods also applies to the asynchronous and span-based forms.)

Example 15-1. The members at the heart of `Stream`

```
public abstract int Read(byte[] buffer, int offset, int count);
public abstract void Write(byte[] buffer, int offset, int count);
public abstract long Position { get; set; }
```

Some streams are read-only. For example, when the input stream for a console application represents the keyboard or the output of some other program, there's no meaningful way for the program to write to that stream. (And for consistency, even if you use input redirection to run a console application with a file as its input, the input stream will be read-only.) Some streams are write-only, such as the output stream of a console application. If you call `Read` on a write-only stream or `Write` on a read-only one, these methods throw a `NotSupportedException`.

TIP

The `Stream` class defines various `bool` properties that describe a stream's capabilities, so you don't have to wait until you get an exception to find out what sort of stream you've got. You can check the `CanRead` or `CanWrite` properties.

Both `Read` and `Write` take a `byte[]` array as their first argument, and these methods copy data into or out of that array, respectively. The `offset` and `count` arguments that follow indicate the array element at which to start and the number of bytes to read or write; you do not have to use the whole array. Notice that there are no arguments to specify the offset within the stream at which to read or write. This is managed by the `Position` property—this starts at zero, but each time you read or write, the position advances by the number of bytes processed.

Notice that the `Read` method returns an `int`. This tells you how many bytes were read from the stream—the method does not guarantee to provide the amount of data you requested. One obvious reason for this is that you could reach the end of the stream, so even though you may have asked to read 100 bytes into your array, there may have been only 30 bytes of data left between the current `Position` and the end of the stream. However, that's not the only reason you might get less than you asked for, and this often catches people out, so for the benefit of people skim-reading this chapter, I'll put this in a scary warning.

WARNING

If you ask for more than one byte at a time, a `Stream` is always free to return less data than you requested from `Read` for any reason. You should never presume that a call to `Read` returned as much data as it could, even if you have good reason to know that the amount you asked for will be available.

The reason `Read` is slightly tricky is that some streams are live, representing a source of information that produces data gradually as the program runs. For example, if a console application is running interactively, its input stream can provide data only as fast as the user types; a stream representing data being received over a network connection can provide data only as fast as it arrives. If you call `Read` and you ask for more data than is currently available, a stream might wait until it has as much as you've asked for, but it doesn't have to—it may return whatever data it has immediately. (The only situation in which it is obliged to wait before returning is if it currently has no data at all but is not yet at the end of the stream. It has to return at least one byte, because a `0` return value indicates the end of the stream.) If you want to ensure that you read a specific number of bytes, you used to have to check whether `Read` returned

fewer bytes than you wanted, and if necessary, keep calling it until you have what you need. (On .NET Framework, you still have to do this.) Fortunately, .NET 7.0 added a new method, `ReadExactly`, which takes the same arguments as `Read`, but won't return until it either reaches the end of the stream, or is able to return as many bytes as you asked for. It also added a similar `ReadAtLeast`, which will never return fewer bytes than you asked for (unless it reaches the end of the stream) but can return more if the stream happens to have more available immediately.

`Stream` also offers a simpler way to read. The `ReadByte` method returns a single byte, unless you hit the end of the stream, at which point it returns a value of `-1`. (Its return type is `int`, enabling it to return any possible value for `byte` as well as negative values.) This avoids the problem of being handed back only some of the data you requested, because if you get anything back at all, you always get exactly one byte. However, it's not especially convenient or efficient if you want to read larger chunks of data.

The `Write` method doesn't have any of these issues. If it succeeds, it always accepts all of the data you provide. Of course, it might fail—it could throw an exception before it manages to write all of the data because of an error (e.g., running out of space on disk or losing a network connection).

Position and Seeking

Streams automatically update their current position each time you read or write. As you can see in [Example 15-1](#), the `Position` property can be set, so you can attempt to move directly to a particular position. This is not guaranteed to work because it's not always possible to support it. For example, a `Stream` that represents data being received over a TCP network connection could produce data indefinitely—as long as the connection remains open and the other end keeps sending data, the stream will continue to honor calls to `Read`. A connection could remain open for many days and might receive terabytes of data in that time. If such a stream let you set its `Position` property, enabling your code to go back and reread data received earlier, the stream would have to find somewhere to store every single byte it received just in case the code using the stream wants to see it again. Since that might involve storing more data than you have space for on disk, this is clearly not practical, so some

streams will throw `NotSupportedException` when you try to set the `Position` property. (There's a `CanSeek` property you can use to discover whether a particular stream supports changing the position, so just like with read-only and write-only streams, you don't have to wait until you get an exception to find out whether it will work.)

As well as the `Position` property, `Stream` also defines a `Seek` method, whose signature is shown in [Example 15-2](#). This lets you specify the position you require relative to the stream's current position. (This also throws `NotSupportedException` on streams that don't support seeking.)

Example 15-2. The `Seek` method

```
public abstract long Seek(long offset, SeekOrigin origin);
```

If you pass `SeekOrigin.Current` as the second argument, it will set the position by adding the first argument to the current position. You can pass a negative `offset` if you want to move backward. You can also pass `SeekOrigin.End` to set the position to be some specified number of bytes from the end of the stream. Passing `SeekOrigin.Begin` has the same logical effect as just setting `Position`—it sets the position relative to the start of the stream.

Flushing

As with many stream APIs on other programming systems, writing data to a `Stream` does not necessarily cause the data to reach its destination immediately. When a call to `Write` returns, all you know is that it has copied your data somewhere; but that might be a buffer in memory, not the final target. For example, if you write a single byte to a stream representing a file on a storage device, the stream object will typically defer writing that to the drive until it has enough bytes to make it worth the effort. Storage devices are block-based, meaning that writes happen in fixed-size chunks, typically several kilobytes in size, so it generally makes sense to wait until there's enough data to fill a block before writing anything out.

This buffering is usually a good thing—it improves write performance while enabling you to ignore the details of how the disk works. However,

a downside is that if you write data only occasionally (e.g., when writing error messages to a logfile), you could easily end up with long delays between the program writing data to a stream and that data reaching the disk. This could be perplexing for someone trying to diagnose a problem by looking at the logfiles of a program that's currently running. And more insidiously, if your program crashes, anything in a stream's buffers that has not yet made it to the storage device will probably be lost.

The `Stream` class therefore offers a `Flush` method. This lets you tell the stream that you want it to do whatever work is required to ensure that any buffered data is written to its target, even if that means making sub-optimal use of the buffer.

When using a `FileStream`, the `Flush` method does not necessarily guarantee that the data being flushed has made it to disk yet. It merely makes the stream pass the data to the OS. Before you call `Flush`, the OS might not even have seen the data, so if you were to terminate the process suddenly, the data would be lost. After `Flush` has returned, the OS has everything your code has written, so the process could be terminated without loss of data. However, the OS may perform additional buffering of its own, so if the power fails before the OS gets around to writing everything to disk, the data will still be lost. If you need to guarantee that data has been written persistently (rather than merely ensuring that you've handed it to the OS), you will also need to either use the `WriteThrough` flag, described in [“FileStream Class”](#), or call the `Flush` overload that takes a `bool`, passing `true` to force flushing to the storage device.

A stream automatically flushes its contents when you call `Dispose`. You need to use `Flush` only when you want to keep a stream open after writing out buffered data. It is particularly important if there will be extended periods during which the stream is open but inactive. (If the stream represents a network connection, and if your application depends on prompt data delivery—this would be the case in an online chat application or game, for example—you would call `Flush` even if you expect only fairly brief periods of inactivity.)

Copying

Copying all of the data from one stream to another is occasionally useful. It wouldn't be hard to write a loop to do this, but you don't have to, be-

cause the `Stream` class's `CopyTo` method (or the equivalent `CopyToAsync`) does it for you. There's not much to say about it. The main reason I'm mentioning it is that it's not uncommon for developers to write their own version of this method because they didn't know the functionality was built into `Stream`.

Length

Some streams are able to report their length through the predictably named `Length` property. As with `Position`, this property's type is `long`—`Stream` uses 64-bit numbers because streams often need to be larger than 2 GB, which would be the upper limit if sizes and positions were represented with `int`.

`Stream` also defines a `SetLength` method that lets you define the length of a stream (where supported). You might think about using this when writing a large quantity of data to a file, to ensure that there is enough space to contain all the data you wish to write—better to get an `IOException` before you start than wasting time on a doomed operation and potentially causing system-wide problems by using up all of the free space. However, many filesystems support sparse files, letting you create files far larger than the available free space, so in practice you might not see any error until you start writing nonzero data. Even so, if you specify a length that is longer than the filesystem supports, `SetLength` will throw an `ArgumentException`.

Not all streams support length operations. The `Stream` class documentation says that the `Length` property is available only on streams that support `CanSeek`. This is because streams that support seeking are typically ones where the whole content of the stream is known and accessible up front. Seeking is unavailable on streams where the content is produced at runtime (e.g., input streams representing user input or streams representing data received over the network), and in those cases the length is also very often not known in advance. As for `SetLength`, the documentation states that this is supported only on streams that support both writing and seeking. (As with all members representing optional features, `Length` and `SetLength` will throw a `NotSupportedException` if you try to use these members on streams that do not support them.)

Disposal

Some streams represent resources external to the .NET runtime. For example, `FileStream` provides stream access to the contents of a file, so it needs to obtain a file handle from the OS. It's important to close handles when you're done with them; otherwise you might prevent other applications from being able to use the file. Consequently, the `Stream` class implements the `IDisposable` interface (described in [Chapter 7](#)) so that it can know when to do that. And, as I mentioned earlier, buffering streams such as `FileStream` flush their buffers when you call `Dispose`, before closing handles.

Not all stream types depend on `Dispose` being called: `MemoryStream` works entirely in memory, so the GC would be able to take care of it. But in general, if you caused a stream to be created, you should call `Dispose` when you no longer need it.

NOTE

There are some situations in which you will be provided with a stream, but it is not your job to dispose it. For example, ASP.NET Core can provide streams to represent data in HTTP requests and responses. It creates these for you and then disposes them after you've used them, so you should not call `Dispose` on them.

Confusingly, the `Stream` class also has a `Close` method. This is an accident of history. The first public beta release of .NET 1.0 did not define `IDisposable`, and C# did not have `using` statements—the keyword was only for `using` directives, which bring namespaces into scope. The `Stream` class needed some way of knowing when to clean up its resources, and since there was not yet a standard way to do this, it invented its own idiom. It defined a `Close` method, which was consistent with the terminology used in many stream-based APIs in other programming systems. `IDisposable` was added before the final release of .NET 1.0, and the `Stream` class added support for this, but it left the `Close` method in place; removing it would have disrupted a lot of early adopters who had been using the betas. But `Close` is redundant, and the documentation actively advises against using it. It says you should call `Dispose` instead (through a `using` statement if that is convenient). There's no harm in

calling `Close`—there’s no practical difference between that and `Dispose`—but `Dispose` is the more common idiom and is therefore preferred.

Asynchronous Operation

The `Stream` class offers asynchronous versions of `Read` and `Write`. Be aware that there are two forms. `Stream` first appeared in .NET 1.0, so it supported what was then the standard asynchronous mechanism, the Asynchronous Programming Model (APM, described in [Chapter 16](#)) through the `BeginRead`, `EndRead`, `BeginWrite`, and `EndWrite` methods. This model is now deprecated, having been superseded by the newer Task-based Asynchronous Pattern (or TAP, also described in [Chapter 16](#)). `Stream` supports this through its `ReadAsync` and `WriteAsync` methods. There are two more operations that did not originally have any kind of asynchronous form that now have TAP versions: `FlushAsync` and `CopyToAsync`. (These support only TAP, because APM was already deprecated by the time Microsoft added these methods.)

WARNING

Avoid the old APM-based `Begin/End` forms of `Read` and `Write`. For a while they were only in .NET Framework. They were later added to .NET Standard 2.0 to make it easier to migrate existing code from .NET Framework to .NET, so they are supported only for legacy scenarios.

Some stream types implement asynchronous operations using very efficient techniques that correspond directly to the asynchronous capabilities of the underlying OS. (`FileStream` does this, as do the various streams .NET can provide to represent content from network connections.) You may come across libraries with custom stream types that do not do this, but even then, the asynchronous methods will be available, because the base `Stream` class can fall back to using multithreaded techniques instead.

One thing you need to be careful of when using asynchronous reads and writes is that a stream only has a single `Position` property. Reads and writes depend on the current `Position` and also update it when they are done, so in general you must avoid starting a new operation before one already in progress is complete. However, if you wish to perform multiple

concurrent read or write operations from a particular file, `FileStream` has special handling for this. If you tell it that you will be using the file in asynchronous mode, operations use the value `Position` has at the start of the operation, and once an asynchronous read or write has started, you are allowed to change `Position` and start another operation without waiting for all the previous ones to complete. But this only applies to `FileStream`, and only when the file is opened in asynchronous mode. Alternatively, instead of using `FileStream`, you could use the `RandomAccess` class, which defines methods for performing read and write operations directly against a file handle. This class's methods all require you pass an argument specifying the position explicitly for each read and write.

.NET offers `IAsyncDisposable`, an asynchronous form of `Dispose`. The `Stream` class implements this, because disposal often involves flushing, which is a potentially slow operation. (This is not available on .NET Framework.)

Concrete Stream Types

The `Stream` class is abstract, so to use a stream, you'll need a concrete derived type. In some situations, this will be provided for you—the ASP.NET Core web framework supplies stream objects representing HTTP request and response bodies, for example, and the client-side `HttpClient` class will do something similar. But sometimes you'll need to create a stream object yourself. This section describes a few of the more commonly used types that derive from `Stream`.

The `FileStream` class represents a file on the filesystem. I will describe this in [“Files and Directories”](#).

`MemoryStream` lets you create a stream on top of a `byte[]` array. You can either take an existing `byte[]` and wrap it in a `MemoryStream`, or you can create a `MemoryStream` and then populate it with data by calling `Write` (or `WriteAsync`). You can retrieve the populated `byte[]` once you're done by calling either `ToArray` or `GetBuffer`. (`ToArray` allocates a new array, with the size based on the number of bytes actually written. `GetBuffer` is more efficient because it returns the underlying array `MemoryStream` is using, but unless the writes happened to fill it completely, the array returned will typically be oversized, with some unused

space at the end.) This class is useful when you are working with APIs that require a stream and you don't have one for some reason. For example, most of the serialization APIs described later in this chapter work with streams, but you might end up wanting to use that in conjunction with some other API that works in terms of `byte[]`. `MemoryStream` lets you bridge between those two representations.

Both Windows and Unix define an interprocess communication (IPC) mechanism enabling you to connect two processes through a stream. Windows calls these *named pipes*. Unix also has a mechanism with that name, but it is completely different; it does, however, offer a mechanism similar to Windows named pipes: *domain sockets*. Although the precise details of Windows named pipes and Unix domain sockets differ, the various classes derived from `PipeStream` provide a common abstraction for both in .NET.

`BufferedStream` derives from `Stream` but also takes a `Stream` in its constructor. It adds a layer of buffering, which is useful if you want to perform small reads or writes on a stream that is designed to work best with larger operations. (You don't need to use this with `FileStream` because that has its own built-in buffering mechanism.)

There are various stream types that transform the contents of other streams in some way. For example, `DeflateStream`, `GZipStream`, and `BrotliStream` implement three widely used compression algorithms. You can wrap these around other streams to compress the data written to the underlying stream or to decompress the data read from it. (These just provide the lowest-level compression service. If you want to work with the popular ZIP format for packages of compressed files, use the `ZipArchive` class. .NET 7.0 added a `TarFile` class for working with the *tar* archive used for similar purposes in Unix.) There's also a class called `CryptoStream`, which can encrypt or decrypt the contents of other streams using any of the wide variety of encryption mechanisms supported in .NET.

One Type, Many Behaviors

As you've now seen, the abstract base class `Stream` gets used in a wide range of scenarios. It is arguably an abstraction that has been stretched a little too thin. The presence of properties such as `CanSeek` that tell you

whether the particular `Stream` you have can be used in a certain way is arguably a symptom of an underlying problem, an example of something known as a *code smell*. .NET streams did not invent this particular one-size-fits-all approach—it was popularized by Unix and the C programming language’s standard library a long time ago. The problem is that when writing code that deals with a `Stream`, you might not know what sort of thing you are dealing with.

There are many different ways to use a `Stream`, but three usage styles come up a lot:

- Sequential access of a sequence of bytes
- Random access, with a presumption of efficient caching
- Access to some underlying capability of a device or system

As you know, not all `Stream` implementations support all three models—if `CanSeek` returns `false`, that rules out the middle option. But what is less obvious is that even when these properties indicate that a capability is available, not all streams support all usage models equally efficiently.

For example, I worked on a project that used a library for accessing files in a cloud-hosted storage service that was able to represent those files with `Stream` objects. This looks convenient because you can pass those to any API that works with a `Stream`. However, it was designed very much for the third style of use in the preceding list: every single call to `Read` (or `ReadAsync`) would cause the library to make an HTTP request to the storage service. We had initially hoped to use this with another library that knew how to parse Parquet files (a binary tabular data storage format widely used in high-volume data processing). However, it turned out that the library was expecting a stream that supported the second type of access: it jumped back and forth through the file, making large numbers of fairly small reads. It worked perfectly well with the `FileStream` type I’ll be describing later, because that supports the first two modes of use well. (For the second style, it relies on the OS to do the caching.) But it would have been a performance disaster to plug a `Stream` from the storage service library directly into the Parquet parsing library.

It’s not always obvious when you have a mismatch of this kind. In this example, the properties that report capabilities (such as `CanSeek`) gave no clue that there would be a problem. And applications that use Parquet

files often use some sort of remote storage service, rather than the local filesystem, so there was no obvious reason to think that this library would presume that any `Stream` would offer local filesystem-like caching. It did technically work when we tried it: the storage library `Stream` worked hard to do everything asked of it, and the code worked correctly...eventually. But it was unusably slow. So whenever you use a `Stream`, it's important to make sure you have fully understood what access patterns it will be subjected to and how efficiently it supports those patterns.

In some cases you might be able to bridge the gap. The `BufferedStream` class can often take a `Stream` designed only for the third usage style mentioned previously and adapt it for the first style of usage. However, there's nothing in the runtime libraries that can add support for the second style of usage to a `Stream` that doesn't already innately support it. (This is typically only available either with streams that represent something already fully in memory or that wrap some local API that does the caching for you, such as the OS filesystem APIs.) In these cases you will either need to rethink your design (e.g., make a local copy of the `Stream` contents), change the way that the `Stream` is consumed, or write some sort of custom caching adapter. (In the end, we wrote an adapter that augmented the capabilities of `BufferedStream` with just enough random access caching to solve the performance problems.)

Text-Oriented Types

`Stream` is byte oriented, but it's common to work with files that contain text. If you want to process text stored in a file (or received over the network), it is cumbersome to use a byte-based API, because this forces you to deal explicitly with all of the variations that can occur. For example, there are multiple conventions for how to represent the end of a line—Windows typically uses two bytes with values of `13` and `10`, as do many internet standards such as HTTP, but Unix-like systems often use just a single byte with the value `10`.

There are also multiple character encodings in popular use. Some files use one byte per character, some use two, and some use a variable-length encoding. There are many different single-byte encodings too, so if you

encounter a byte value of, say, 163 in a text file, you cannot know what that means unless you know which encoding is in use.

In a file using the single-byte Windows-1252 encoding, the value 163 represents a pound sign: £.¹ But if the file is encoded with ISO/IEC 8859-5 (designed for regions that use Cyrillic alphabets), the exact same code represents the Cyrillic capital letter DJE: Ъ. And if the file uses the UTF-8 encoding, the value 163 would only be allowed as part of a multibyte sequence representing a single character.

Awareness of these issues is, of course, an essential part of any developer's skill set, but that doesn't mean you should have to handle every little detail any time you encounter text. So .NET defines specialized abstractions for working with text.

TextReader and TextWriter

The abstract `TextReader` and `TextWriter` classes present data as a sequence of `char` values. Logically speaking, these classes are similar to a stream, but each element in the sequence is a `char` instead of a `byte`. However, there are some differences in the details. For one thing, there are separate abstractions for reading and writing. `Stream` combines these, because it's common to want read/write access to a single entity, particularly if the stream represents a file on disk. For byte-oriented random access, this makes sense, but it's a problematic abstraction for text.

Variable-length encodings make it tricky to support random write access (i.e., the ability to change values at any point in the sequence). Consider what it would mean to take a 1 GB UTF-8 text file whose first character is a \$ and replace that first character with a £. In UTF-8, the \$ character takes only one byte, but £ requires two, so changing that first character would require an extra byte to be inserted at the start of the file. This would mean moving the remaining file contents—almost 1 GB of data—along by one byte.

Even read-only random access is relatively expensive. Finding the millionth character in a UTF-8 file requires you to read the first 999,999 characters, because without doing that, you have no way of knowing what mix of single-byte and multibyte characters there is. The millionth character might start at the millionth byte, but it could also start some four

million bytes in, or anywhere in between. Since supporting random access with variable-length text encodings is expensive, particularly for writable data, these text-based types don't offer it. Without random access, there's no real benefit in merging readers and writers into one type. Also, separating reader and writer types removes the need to check the `CanWrite` property—you know that you can write because you've got a `TextWriter`.

`TextReader` offers several ways to read data. The simplest is the zero-argument overload of `Read`, which returns an `int`. This will return `-1` if you've reached the end of the input and will otherwise return a character value. (You'll need to cast it to a `char` once you've verified that it's non-negative.) Alternatively, there are two methods that look similar to the `Stream` class's `Read` method, as [Example 15-3](#) shows.

Example 15-3. `TextReader` chunk reading methods

```
public virtual int Read(char[] buffer, int index, int count) {...}  
public virtual int ReadBlock(char[] buffer, int index, int count) {...}
```

Just like `Stream.Read`, these take an array, as well as an index into that array and a count, and will attempt to read the number of values specified. The most obvious difference from `Stream` is that these use `char` instead of `byte`. But what's the difference between `Read` and `ReadBlock`? `ReadBlock` solves the same problem as `ReadExactly` does for streams: whereas `Read` may return fewer characters than you asked for, `ReadBlock` will not return until either as many characters as you asked for are available or it reaches the end of the content.

One of the challenges of handling text input is dealing with the various conventions for line endings, and `TextReader` can insulate you from that. Its `ReadLine` method reads an entire line of input and returns it as a `string`. This string will not include the end-of-line character or characters.

NOTE

`TextReader` does not presume one particular end-of-line convention. It accepts either a carriage return (character value 13, which we write as `\r` in string literals) or a line feed (10, or `\n`). And if both characters appear adjacently, the character pair is treated as being a single end of line, despite being two characters. This processing happens only when you use either `ReadLine` or `ReadLineAsync`. If you work directly at the character level by using `Read` or `ReadBlock`, you will see the end-of-line characters exactly as they are.

`TextReader` also offers `ReadToEnd`, which reads the input in its entirety and returns it as a single `string`. And finally, there's `Peek`, which does the same thing as the single-argument `Read` method, except it does not change the state of the reader. It lets you look at the next character without consuming it, so the next time you call either `Peek` or `Read`, it will return the same character again.

As for `TextWriter`, it offers two overloaded methods for writing: `Write` and `WriteLine`. Each of these offers overloads for all of the built-in value types (`bool`, `int`, `float`, etc.). Functionally, the class could have gotten away with a single overload that takes an `object`, because that can just call `ToString` on its argument, but these specialized overloads make it possible to avoid boxing the argument. `TextWriter` also offers a `Flush` method for much the same reason that `Stream` does.

By default, a `TextWriter` will use the default end-of-line sequence for the OS you are running on. On Windows this is the `\r\n` sequence (13, then 10). On Linux or macOS you will just get a single `\n` at each line end. You can change this by setting the writer's `NewLine` property.

Both of these abstract classes implement `IDisposable` because some of the concrete derived text reader and writer types are wrappers around other disposable resources.

As with `Stream`, these classes offer asynchronous methods for reading and writing (although `StreamReader` doesn't implement `IAsyncDisposable`). Unlike with `Stream`, this was a fairly recent addition, so they support only the task-based pattern described in [Chapter 16](#), which can be consumed with the `await` keyword described in [Chapter 17](#).

Concrete Reader and Writer Types

As with `Stream`, various APIs in .NET will present you with `TextReader` and `TextWriter` objects. For example, the `Console` class defines `In` and `Out` properties that provide textual access to the process's input and output streams. I've not described these before, but we have been using them implicitly—the `Console.WriteLine` method overloads are all just wrappers that call `Out.WriteLine` for you. Likewise, the `Console` class's `Read` and `ReadLine` methods simply forward to `In.Read` and `In.ReadLine`. There's also `Error`, another `TextWriter` for writing to the standard error output stream. However, there are some concrete classes that derive from `TextReader` or `TextWriter` that you might want to instantiate directly.

StreamReader and StreamWriter

Perhaps the most useful concrete text reader and writer types are `StreamReader` and `StreamWriter`, which wrap a `Stream` object. You can pass a `Stream` as a constructor argument, or you can just pass a string containing the path of a file, in which case they will automatically construct a `FileStream` for you and then wrap that. [Example 15-4](#) uses this technique to write some text to a file.

Example 15-4. Writing text to a file with `StreamWriter`

```
using (var fw = new StreamWriter(@"c:\temp\out.txt"))
{
    fw.WriteLine($"Writing to a file at {DateTime.Now}");
}
```

There are various constructor overloads offering more fine-grained control. When passing a string in order to use a file with a `StreamWriter` (as opposed to some `Stream` you have already obtained), you can optionally pass a `bool` indicating whether to start from scratch or to append to an existing file if one exists. (A `true` value enables appending.) If you do not pass this argument, appending is not used, and writing will begin from the start. You can also specify an encoding. By default, `StreamWriter` will use UTF-8 with no byte order mark (BOM), but you can pass any type derived from the `Encoding` class, which is described in [“Encoding”](#).

`StreamReader` is similar—you can construct it by passing either a `Stream` or a `string` containing the path of a file, and you can optionally specify an encoding. However, if you don't specify an encoding, the behavior is subtly different from `StreamWriter`. Whereas `StreamWriter` just defaults to UTF-8, `StreamReader` will attempt to detect the encoding from the stream's content. It looks at the first few bytes and will look for certain features that are typically a good sign that a particular encoding is in use. If the encoded text begins with a Unicode BOM, this makes it possible to determine with high confidence what the encoding is.

StringReader and StringWriter

The `StringReader` and `StringWriter` classes serve a similar purpose to `MemoryStream`: they are useful when you are working with an API that requires either a `TextReader` or `TextWriter`, but you want to work entirely in memory. Whereas `MemoryStream` presents a `Stream` API on top of a `byte[]` array, `StringReader` wraps a `string` as a `TextReader`, while `StringWriter` presents a `TextWriter` API on top of a `StringBuilder`.

One of the APIs .NET offers for working with XML, `XmlReader`, requires either a `Stream` or a `TextReader`. Suppose you have XML content in a `string`. If you pass a `string` when creating a new `XmlReader`, it will interpret that as a URI from which to fetch the content, rather than the content itself. The constructor for `StringReader` that takes a string just wraps that string as the content of the reader, and we can pass that to the `XmlReader.Create` overload that requires a `TextReader`, as [Example 15-5](#) shows. (The line that does this is in bold—the code that follows just uses the `XmlReader` to read the content to show that it works as expected.)

Example 15-5. Wrapping a string in a `StringReader`

```
string xmlContent =
    "<message><text>Hello</text><recipient>world</recipient></message>";
var xmlReader = XmlReader.Create(new StringReader(xmlContent));
while (xmlReader.Read())
{
    if (xmlReader.NodeType == XmlNodeType.Text)
    {
        Console.WriteLine(xmlReader.Value);
    }
}
```

```
}  
}
```

`StringWriter` is even simpler: you can just construct it with no arguments. Once you've finished writing to it, you can call either `ToString` or `GetStringBuilder` to extract all of the text that has been written.

Encoding

As I mentioned earlier, if you're using the `StreamReader` or `StreamWriter`, these need to know which character encoding the underlying stream uses to be able to convert correctly between the bytes in the stream and .NET's `char` or `string` types. To manage this, the `System.Text` namespace defines an abstract `Encoding` class, with various encoding-specific public concrete derived types, including `ASCIIEncoding`, `UTF7Encoding`, `UTF8Encoding`, `UTF32Encoding`, and `UnicodeEncoding`.

Most of those type names are self-explanatory, because they are named after the standard character encodings they represent, such as ASCII or UTF-8. The one that requires a little more explanation is `UnicodeEncoding`—after all, UTF-7, UTF-8, and UTF-32 are all Unicode encodings, so what's this other one for? When Windows introduced support for Unicode back in the first version of Windows NT, it adopted a slightly unfortunate convention: in documentation and various API names, the term *Unicode* was used to refer to a 2-byte little-endian² character encoding, which is just one of many possible encoding schemes, all of which could correctly be described as being “Unicode” of one form or another.

The `UnicodeEncoding` class is named to be consistent with this historical convention, although even then it's still a bit confusing. The encoding referred to as “Unicode” in Win32 APIs is effectively UTF-16LE, but the `UnicodeEncoding` class is also capable of supporting the big-endian UTF-16BE.

The base `Encoding` class defines static properties that return instances of all the encoding types I've mentioned, so if you need an object representing a particular encoding, you would normally just write `Encoding.ASCII` or `Encoding.UTF8`, etc., instead of constructing a new object. There are two properties of type `UnicodeEncoding`: the `Unicode`

property returns one configured for UTF-16LE, and `BigEndianUnicode` returns one for UTF-16BE.

For the various Unicode encodings, these properties will return encoding objects that will tell `StreamWriter` to generate a byte order mark at the start of the output. The main purpose of the BOM is to enable software that reads encoded text to detect automatically whether the encoding is big- or little-endian. (You can also use it to recognize UTF-8, because that encodes the BOM differently than other encodings.) If you know that you will be using an endian-specific encoding (e.g., UTF-16LE), the BOM is unnecessary, because you already know the order, but the Unicode specification defines adaptable formats in which the encoded bytes can advertise the order in use by starting with a BOM, a character with Unicode code point `U+FEFF`. The 16-bit version of this encoding is just called UTF-16, and you can tell whether any particular set of UTF-16-encoded bytes is big- or little-endian by seeing whether it begins with `0xFE`, `0xFF` or `0xFF`, `0xFE`.

WARNING

Although Unicode defines encoding schemes that allow the endianness to be detected, it is not possible to create an `Encoding` object that works that way—it will always have a specific endianness. So, although an `Encoding` specifies whether a BOM should be written when writing data, this does not influence the behavior when reading data—it will always presume the endianness specified when the `Encoding` was constructed. This means that the `Encoding.UTF32` property is arguably misnamed—it always interprets data as little-endian even though the Unicode specification allows UTF-32 to use either big- or little-endian.

`Encoding.UTF32` is really UTF-32LE.

As mentioned earlier, if you do not specify an encoding when creating a `StreamWriter`, it defaults to UTF-8 with no BOM, which is different from `Encoding.UTF8`—that will generate a BOM. And recall that `StreamReader` is more interesting: if you do not specify an encoding, it will attempt to detect the encoding. So .NET is able to handle automatic detection of byte ordering as required by the Unicode specification for UTF-16 and UTF-32; it is just that the way to do it is *not* to specify any particular encoding when constructing a `StreamReader`. It will look for a BOM, and if it finds one present, it will use a suitable Unicode encoding; otherwise, it presumes UTF-8 encoding.

UTF-8 is a popular encoding. If your main language is English, it's a particularly convenient representation, because if you happen to use only the characters available in ASCII, each character will occupy a single byte, and the encoded text will have the exact same byte values as it would with ASCII encoding. But unlike ASCII, you're not limited to a 7-bit character set. All Unicode code points are available; you just have to use multi-byte representations for anything outside of the ASCII range. However, although it's very widely used, UTF-8 is not the only popular 8-bit encoding.

Code page encodings

Windows, like DOS before it, has long supported 8-bit encodings that extend ASCII. ASCII is a 7-bit encoding, meaning that with 8-bit bytes you have 128 “spare” values to use for other characters. This is nowhere near enough to cover every character for every locale, but within a particular country, it's often enough to get by (although not always—many Far Eastern countries need more than 8 bits per character). But each country tends to want a different set of non-ASCII characters, depending on which accented characters are popular in that locale and whether a non-Roman alphabet is required. So various *code pages* exist for different locales. For example, code page 1253 uses values in the range 193–254 to define characters from the Greek alphabet (filling the remaining non-ASCII values with useful characters such as non-US currency symbols). Code page 1255 defines Hebrew characters instead, while 1256 defines Arabic characters in the upper range (and there is some common ground for these particular code pages, such as using 128 for the euro symbol, €, and 163 for the pound sign, £).

One of the most commonly encountered code pages is 1252, because that's the Windows default for English-speaking locales. This does not define a non-Roman alphabet; instead it uses the upper character range for useful symbols and for various accented versions of the Roman alphabet that enable a wide range of Western European languages to be adequately represented.

You can create an encoding for a code page by calling the `Encoding.GetEncoding` method, passing in the code page number. (The concrete type of the object you get back is often not one of those I listed earlier. This method may return nonpublic types that derive from

Encoding.) [Example 15-6](#) uses this to write text containing a pound sign to a file using code page 1252.

Example 15-6. Writing with the Windows 1252 code page

```
using (var sw = new StreamWriter("Text.txt", false,
                                Encoding.GetEncoding(1252)))
{
    sw.Write("£100");
}
```

This will encode the £ symbol as a single byte with the value 163. With the default UTF-8 encoding, it would have been encoded as two bytes, with values of 194 and 163, respectively.

Using encodings directly

`TextReader` and `TextWriter` are not the only way to use encodings. Objects representing encodings (such as `Encoding.UTF8`) define various members. The `GetBytes` method converts a `string` directly to a `byte[]` array, for example, and the `GetString` method converts back again.

You can also discover how much data these conversions will produce. `GetByteCount` tells you how large an array `GetBytes` would produce for a given string, while `GetCharCount` tells you how many characters decoding a particular array would generate. You can also find an upper limit for how much space will be required without knowing the exact text with `GetMaxByteCount`. Instead of a `string`, this takes a number, which it interprets as a string length; since .NET strings use UTF-16, this means that this API answers the question “If I have this many UTF-16 code units, what’s the largest number of code units that might be required to represent the same text in the target encoding?” This can produce a significant overestimate for variable-length encodings. For example, with UTF-8, `GetMaxByteCount` multiplies the length of the input string by three³ and adds an extra 3 bytes to deal with an edge case that can occur with surrogate characters. It produces a correct description of the worst possible case, but text containing any characters that don’t require 3 bytes in UTF-8 (i.e., any text in English or any other languages that use the Latin alphabet, and also any text using Greek, Cyrillic, Hebrew, or Arabic writing sys-

tems, for example) will require significantly less space than `GetMaxByteCount` predicts.

Some encodings can provide a *preamble*, a distinctive sequence of bytes that, if found at the start of some encoded text, indicates that you are likely to be looking at something using that encoding. This can be useful if you are trying to detect which encoding is in use when you don't already know. The various Unicode encodings all return their encoding of the BOM as the preamble, which you can retrieve with the `GetPreamble` method.

The `Encoding` class defines instance properties offering information about the encoding. `EncodingName` returns a human-readable name for the encoding, but there are two more names available. The `WebName` property returns the standard name for the encoding registered with the Internet Assigned Numbers Authority (IANA), which manages standard names and numbers for things on the internet such as MIME types. Some protocols, such as HTTP, sometimes put encoding names into headers, and this is the text you should use in that situation. The other two names, `BodyName` and `HeaderName`, are somewhat more obscure and are used only for internet email—there are different conventions for how certain encodings are represented in the body and headers of email.

Files and Directories

The abstractions I've shown so far in this chapter are very general purpose in nature—you can write code that uses a `Stream` without needing to have any idea where the bytes it contains come from or are going to, and likewise, `TextReader` and `TextWriter` do not demand any particular origin or destination for their data. This is useful because it makes it possible to write code that can be applied in a variety of scenarios. For example, the stream-based `GZipStream` can compress or decompress data from a file, over a network connection, or from any other stream.

However, there are occasions where you know you will be dealing with files and want access to file-specific features. This section describes the classes for working with files and the filesystem.

FileStream Class

The `FileStream` represents a file from the filesystem. I've used it a few times in passing already. It derives from `Stream`, so the preceding sections have already described most of what you need to know except for one aspect: when it comes to creating a `FileStream`, it offers several constructors offering a great deal of control.

We can supply various pieces of information when creating a new `FileStream`. [Table 15-1](#) lists the various types you can supply to the constructor in addition to the path.⁴ The most flexible `FileStream` constructor takes the file path and a `FileStreamOptions`, which supports all possible settings. However, `FileStream` offers other constructor overloads taking various combinations of the types in that table, to simplify certain common scenarios.

Table 15-1. `FileStream` constructor argument types

Type	Purpose	Default
<code>FileMode</code>	Determines the behavior if the file already exists, or does not exist	Not applicable
<code>FileAccess</code>	Indicates whether we will we be reading, writing, or both	<code>ReadWrite</code> (Write if using <code>FileMode.Append</code>)
<code>FileShare</code>	Specifies our tolerance for other applications using the file at the same time	<code>FileShare.Read</code>
<code>FileOptions</code>	Various usage settings including async and cache write-through	<code>FileOptions.None</code>
<code>FileStreamOptions</code>	Combines all preceding settings and some less common ones	Not applicable

`FileMode` is normally⁵ nonoptional, because `FileStream` needs to know whether you're expecting to create a new file, or open an existing one.

[Table 15-2](#) shows the behaviors you can choose between.

Table 15-2. `FileMode` enumeration

Value	Behavior if file exists	Behavior if file does not exist
<code>CreateNew</code>	Throws <code>IOException</code>	Creates new file
<code>Create</code>	Replaces existing file	Creates new file
<code>Open</code>	Opens existing file	Throws <code>FileNotFoundException</code>
<code>OpenOrCreate</code>	Opens existing file	Creates new file
<code>Truncate</code>	Replaces existing file	Throws <code>FileNotFoundException</code>
<code>Append</code>	Opens existing file, setting <code>Position</code> to end of file	Creates new file

The `FileAccess` values of `Read`, `Write`, and `ReadWrite` are self-explanatory, but `FileShare` is more subtle. It doesn't state how we intend to use the file; it states how other processes should be allowed to use the file while we have it open. For example, when using `FileAccess.Write`, we might specify `FileShare.Read`, indicating that we're happy for other processes to read from the file, but we want to be the only ones writing. `FileShare.None` demands exclusive access.

The `FileShare` comes into play when we try to open the file—the constructor will throw an `IOException` if the file is already open elsewhere in a way that conflicts with our stated requirements. (There are two kinds of conflict: our specified `FileShare` may be incompatible with `FileAccess` with which the file is already open elsewhere—perhaps we've said `FileShare.Read` but some other process already has the file open with `FileAccess.Write`. Or our specified `FileAccess` may be incompatible with the `FileShare` with which the file is already open. Perhaps we're asking for `FileAccess.Read`, but some other process has the file open with `FileShare.None`.) If we successfully open the file, then for as long as we keep it open, the `FileAccess` and `FileShare` we specified impose constraints on any other attempts to open the same file.

WARNING

Unix has fewer file-locking mechanisms than Windows, so these sharing semantics will often be mapped to something simpler on Linux and macOS. Also, file locks are advisory in Unix, meaning processes can simply ignore them if they want to.

While `FileStream` gives you control over the contents of the file, some operations you might wish to perform on files are either cumbersome or not supported at all with `FileStream`. For example, you can copy a file with this class, but it's not as straightforward as it could be, and `FileStream` does not offer any way to delete a file. So the runtime libraries include a separate class for these kinds of operations.

File Class

The static `File` class provides methods for performing various operations on files. For example, the `Delete` method removes the named file from the filesystem. The `Move` method can either move or just rename a file. There are methods for retrieving information and attributes that the filesystem stores about each file, such as `GetCreationTime`, `GetLastAccessTime`, `GetLastWriteTime`, and `GetAttributes`. (The last of those returns a `FileAttributes` value, which is a flags enumeration type telling you whether the file is read only, a hidden file, a system file, and so on.) .NET 7.0 added `Get/SetUnixFileMode` methods to work with the mode flags present in Unix filesystems.

The `Exists` method lets you discover whether a file exists before you attempt to open it. You don't strictly need this, because `FileStream` will throw a `FileNotFoundException` exception if you attempt to open a nonexistent file, but `Exists` is useful if you don't need to do anything with the file other than determining whether it is there. If you are planning to open the file anyway, and are just trying to avoid an exception, you should be wary of this method; just because `Exists` returns `true`, that's no guarantee that you won't get a `FileNotFoundException` exception. It's always possible that in between your checking for a file's existence and attempting to open it, another process might delete the file. Alternatively, the file might be on a network share, and you might lose network connectivity. So you

should always be prepared for exceptions with file access, even if you've attempted to avoid provoking them.

`File` offers many helper methods to simplify opening or creating files. The `Create` method simply constructs a `FileStream` for you, passing in suitable `FileMode`, `FileAccess`, and `FileShare` values. [Example 15-7](#) shows how to use it and also shows what the equivalent code would look like without using the `Create` helper. The `Create` method provides overloads letting you specify the buffer size, `FileOptions`, and `FileSecurity`, but these still provide the other arguments for you.

Example 15-7. `File.Create` versus new `FileStream`

```
using (FileStream fs = File.Create("foo.bar"))
{
    ...
}

// Equivalent code without using File class
using (var fs = new FileStream("foo.bar", FileMode.Create,
                             FileAccess.ReadWrite, FileShare.None))
{
    ...
}
```

The `File` class's `OpenRead` and `OpenWrite` methods provide similar decluttering for when you want to open an existing file for reading or open or create a file for writing. It also offers several text-oriented helpers, such as `File.AppendAllText`, which appends all of the text in a string to a file. This has the same effect as opening the file, wrapping it in a `StreamWriter`, writing the text, then closing the file again, but as [Example 15-8](#) shows, it reduces all that to a single method call.

Example 15-8. Appending a single string to a file

```
File.AppendAllText(@"c:\temp\log.txt", message);
```

Be careful, though. This does not automatically add any end-of-line characters, so if you were to call `AppendAllText` multiple times, you'd end up with one long line in your output file, unless the strings you were using happened to contain end-of-line characters. If you want to work with

lines, there's an `AppendAllLines` method that takes a collection of strings and appends each as a new line to the end of a file. [Example 15-9](#) uses this to append a full line with each call.

Example 15-9. Appending a single line to a file

```
static void Log(string message)
{
    File.AppendAllLines(@"c:\temp\log.txt", new[] { message });
}
```

Since `AppendAllLines` accepts an `IEnumerable<string>`, you can use it to append any number of lines. But it's perfectly happy to append just one if that's what you want. `File` also defines `WriteAllText` and `WriteAllLines` methods, which work in a very similar way, but if there is already a file at the specified path, these will replace it instead of appending to it.

There are also some related text-oriented methods for reading the contents of files. `ReadAllText` performs the equivalent of constructing a `StreamReader` and then calling its `ReadToEnd` method—it returns the entire content of the file as a single `string`. `ReadAllBytes` fetches the whole file into a `byte[]` array. `ReadAllLines` reads the whole file as a `string[]` array, with one element for each line in the file. `ReadLines` is superficially very similar. It provides access to the whole file as an `IEnumerable<string>` with one item for each line, but the difference is that it works lazily—unlike all the other methods I've described in this paragraph, it does not read the entire file into memory up front, so `ReadLines` would be a better choice for very large files. It not only consumes less memory, but it also enables your code to get started more quickly—you can begin to process data as soon as the first line can be read from disk, whereas none of the other methods return until they have read the whole file.

Directory Class

Just as `File` is a static class offering methods for performing operations with files, `Directory` is a static class offering methods for performing operations with directories. Some of the methods are very similar to those offered by `File`—there are methods to get and set the creation time, last

access time, and last write time, for example, and we also get `Move`, `Exists`, and `Delete` methods. The latter has an overload taking a `bool` that, if `true`, will delete everything in the folder, recursively deleting any nested folders and the files they contain. Use that one carefully.

Of course, there are also directory-specific methods. `GetFiles` takes a directory path and returns a `string[]` array containing the full path of each file in that directory. Similarly, `GetDirectories` returns the directories inside the specified directory instead of the files.

`GetFileSystemEntries` returns both files and folders. Each of these offers an overload that lets you specify a pattern by which to filter the results, and a third overload that takes a pattern and also a flag that lets you request recursive searching of all subfolders.

There are also methods called `EnumerateFiles`, `EnumerateDirectories`, and `EnumerateFileSystemEntries`, which do exactly the same thing as the three methods in the preceding paragraph, but they return `IEnumerable<string>`. This is a lazy enumeration, so you can start processing results immediately instead of waiting for all the results as one big array.

You can create new directories too. The `CreateDirectory` method takes a path and will attempt to create as many directories as are necessary to ensure that the path exists. So, if you pass `C:\new\dir\here`, and there is no `C:\new` directory, it will create three new directories: first it will create `C:\new`, then `C:\new\dir`, and then `C:\new\dir\here`. If the folder you ask for already exists, it doesn't treat that as an error; it just returns without doing anything.

Path Class

The static `Path` class provides useful utilities for strings containing file names. Some extract pieces from a file path, such as the containing folder name or the file extension. Some combine strings to produce new file paths. Most of these methods just perform specialized string processing and do not require the files or directories to which the paths refer to exist. However, there are a few that go beyond string manipulation. For example, `Path.GetFullPath` will take the current directory into account if you do not pass an absolute path as the argument. But only the methods that need to make use of real locations will do so.

The `Path.Combine` method deals with the fiddly issues around combining folder and filenames. If you have a folder name, `C:\temp`, and a filename, `log.txt`, passing both to `Path.Combine` returns `C:\temp\log.txt`. And it will also work if you pass `C:\temp\` as the first argument, so one of the issues it deals with is working out whether it needs to supply an extra `\` character. If the second path is absolute, it detects this and simply ignores the first path, so if you pass `C:\temp` and `C:\logs\log.txt`, the result will be `C:\logs\log.txt`. Although these may seem like trivial matters, it's surprisingly easy to get the file path combination wrong if you try to do it yourself by concatenating strings, so you should always avoid the temptation to do that and just use `Path.Combine`.

`Path` has platform-specific behavior. On Unix-like systems, only the `/` character is used as a directory separator, so the various methods in `Path` that expect paths to contain directories will treat only `/` as a separator on these systems. Windows uses a `\` as a separator, although it is common for `/` to be tolerated as a substitute, and `Path` follows suit. So `Path.Combine("/x/y", "/z.txt")` will produce the same results on Windows and Linux, but `Path.Combine(@"\x\y", @"z.txt")` will not. Also, on Windows, if a path begins with a drive letter, it is an absolute path, but Unix does not recognize drive letters. The examples in the preceding paragraph will produce strange-looking results on Linux or macOS because on those systems, all the paths will be treated as relative paths. If you remove the drive letters and replace `\` with `/`, the results will be as you'd expect.

Serialization

The `Stream`, `TextReader`, and `TextWriter` types provide the ability to read and write data in files, networks, or anything else stream-like that provides a suitable concrete class. But these abstractions support only byte or text data. Suppose you have an object with several properties of various types, including some numeric types and perhaps also references to other objects, some of which might be collections. What if you wanted to write all the information in that object out to a file or over a network connection so that an object of the same type and with the same property values could be reconstituted at a later date, or on another computer at the other end of a connection?

You could do this with the abstractions shown in this chapter, but it would require a fair amount of work. You'd have to write code to read each property and write its value out to a `Stream` or `TextWriter`, and you'd need to convert the value to either binary or text. You'd also need to decide on your representation—would you just write values out in a fixed order, or would you come up with a scheme for writing name/value pairs so that you're not stuck with an inflexible format if you need to add more properties later on? You'd also need to come up with ways to handle collections and references to other objects, and you'd need to decide what to do in the face of circular references—if two objects each refer to one another, naive code could end up getting stuck in an infinite loop.

.NET offers several solutions to this problem, each making varying trade-offs between the complexity of the scenarios they are able to support, how well they deal with versioning, and how suitable they are for inter-operating with other platforms. These techniques all fall under the broad name of *serialization* (because they involve writing an object's state into some form that stores data sequentially—serially—such as a `Stream`). Many different mechanisms have been introduced over the years in .NET, so I won't cover all of them. I'll just present the ones that best represent particular approaches to the problem.

BinaryReader, BinaryWriter, and BinaryPrimitives

No discussion of this area is complete without covering the `BinaryReader` and `BinaryWriter` classes, because they solve a fundamental problem that any attempt to serialize and deserialize objects must deal with: they can convert the CLR's intrinsic types to and from streams of bytes. `BinaryPrimitives` does the same thing, but it is able to work with `Span<byte>` and related types, which are discussed in [Chapter 18](#).

`BinaryWriter` is a wrapper around a writable `Stream`. It provides a `Write` method that has overloads for all of the intrinsic types except for `object`. So it can take a value of any of the numeric types, or the `string`, `char`, or `bool` types, and it writes a binary representation of that value into a `Stream`. It can also write arrays of type `byte` or `char`.

`BinaryReader` is a wrapper around a readable `Stream`, and it provides various methods for reading data, each corresponding to the overloads of

Write provided by `BinaryWriter`. For example, you have `ReadDouble`, `ReadInt32`, and `ReadString`.

To use these types, you would create a `BinaryWriter` when you want to serialize some data, and write out each value you wish to store. When you later want to deserialize that data, you'd wrap a `BinaryReader` around a stream containing the data written with the writer, and call the relevant read methods in the exact same order that you wrote the data out in the first place.

`BinaryPrimitives` works slightly differently. It is designed for code that needs to minimize the number of heap allocations, so it's not a wrapper type—it is a static class offering a wide range of methods, such as `ReadInt32LittleEndian` and `WriteUInt16BigEndian`. These take `ReadOnlySpan<byte>` and `Span<byte>` arguments, respectively, because it is designed to work directly with data wherever it may lie in memory (not necessarily wrapped in a `Stream`). However, the basic principle is the same: it converts between byte sequences and primitive .NET types. (Also, string handling is rather more complex: there's no `ReadString` method because anything that returns a `string` will create a new string object on the heap, unless there's a fixed set of possible strings that you can preallocate and hand out again and again. See [Chapter 18](#) for more information about span types.)

These classes only solve the problem of how to represent various built-in types in binary. You are still left with the task of working out how to represent whole objects and what to do about more complex kinds of structures such as references between objects.

CLR Serialization

CLR serialization is, as the name suggests, built into the runtime itself—it is not simply a library feature. You should not use it. CLR serialization has been deprecated for a long time. Recent versions of .NET have disabled the functionality by default. (To be precise, certain types central to the operation of CLR serialization throw exceptions unless you add settings to re-enable them. This has the effect of preventing you from using CLR serialization by default, even though the underlying support mechanisms still exist in the CLR.) Microsoft has announced that in .NET 9.0 it will remove the settings that enabled you to switch it back on.

So why am I telling you about a feature that is deprecated, unavailable by default, and destined to be removed entirely? It's because it offered a useful, distinctive capability. This meant it was widely used, and it can still be tempting to use it. (This will be possible even with .NET 9.0, because the plan is to provide a NuGet package that continues to make it available.) CLR serialization can seem like a good idea, so it's important to know why not to use it.

The most interesting aspect of CLR serialization is that it deals directly with object references. If you serialize, say, a `List<SomeType>` where multiple entries in the list refer to the same object, CLR serialization will detect this, storing just one copy of that object, and when deserializing, it will re-create that one-object-many-references structure. This avoids duplication and also means circular references don't cause problems. (Serialization systems based on the very widely used JSON format normally don't preserve references, although the `JsonSerializer` described later does provide an option to do this.) Moreover, it supports cases where derived types are present—if your `List<SomeType>` contains a mixture of types all derived from `SomeType`, CLR reflection detects this. When it deserializes the list, it will create objects of the same type that were in the original list. It's also simple to use: you just apply a single attribute (`[Serializable]`) to a type, and the CLR automatically handles the process of writing out all of your object's fields during serialization, and reading them back in when deserializing.

This is powerful and easy to use. Why wouldn't we want it?

It's almost impossible to avoid security problems with this style of serialization. This is a consequence of the basic approach: the features I just described mean that a serialized stream describes the types of objects to create, and the values of all the fields in those objects. If you build a system that accepts serialized streams as inputs (e.g., over the network) you are enabling whoever generates those streams to create objects of any type and configuration inside your service. Security research has shown that this provides a springboard from which attackers can run whatever code they like, meaning you have effectively lost control of your system. (This is not unique to .NET by the way. Other platforms have had their own versions of this problem.) The documentation states that CLR serialization's `BinaryFormatter` “is insecure and can't be made secure,” and you will see deprecation warnings when you attempt to use it. So I'm

only describing CLR serialization here because it still gets used despite Microsoft's attempts to end it. (There's also one technical curiosity that exists as a result of CLR serialization. An assumption you might otherwise have made about object creation—specifically that a reference type can only be created through one of its constructors or via `MemberwiseClone`—turns out not be true, because objects can come into existence through deserialization without their constructors running.)

JSON

The JavaScript Object Notation, JSON, is a very widely used serialization format, and the .NET runtime libraries provide support for working with it in the `System.Text.Json` namespace.⁷ It provides three ways of working with JSON data.

The `Utf8JsonReader` and `Utf8JsonWriter` types are stream-like abstractions that represent the contents of JSON data as a sequence of elements. These can be useful if you need to process JSON documents that are too large to load into memory as a single object. They are built on the memory-efficient mechanisms described in [Chapter 18](#), which includes a full example showing how to process JSON with these types. This is a very high-performance option, but it is not the easiest to use.

NOTE

As the names suggest, these types read and write JSON using UTF-8 encoding. This is by far the most widely used encoding for sending and storing JSON, so all of `System.Text.Json` is optimized for it. Because of this, performance-sensitive code should typically avoid ever obtaining a JSON document as a .NET `string`, because that uses UTF-16 encoding and will require conversion to UTF-8 before you can work with these APIs.

There's also the `JsonSerializer` class, which converts between entire .NET objects and JSON. It requires you to define classes with a structure corresponding to the JSON.

Finally, `System.Text.Json` offers types that can provide a description of a JSON document's structure. These are useful when you do not know at development time exactly what the structure of your JSON data will be,

because they provide a flexible object model that can adapt to any shape of JSON data. In fact, there are two variations on this approach. We have `JsonDocument`, `JsonElement`, and related types, which provide a highly efficient read-only mechanism for inspecting a JSON document, and the more flexible but slightly less efficient `JsonNode`, which is writable, enabling you either to build up a description of JSON from scratch or to read in some JSON and then modify it.

JsonSerializer

`JsonSerializer` offers an attribute-driven serialization model in which you define one or more classes mirroring the structure of the JSON data you need to deal with, and can then convert JSON data to and from that model.

[Example 15-10](#) shows a simple model suitable for use with `JsonSerializer`. As you can see, I'm not required to use any particular base class, and there are no mandatory attributes. This example uses the `required` keyword (new in C# 11.0) to indicate that we expect all of the properties to be set. `JsonSerializer` recognizes this, and will throw an exception if you try to deserialize JSON where a `required` property is missing.

Example 15-10. Simple JSON serialization model

```
public class SimpleData
{
    public required int Id { get; set; }
    public required IList<string> Names { get; set; }
    public required NestedData Location { get; set; }
    public required IDictionary<string, int> Map { get; set; }
}

public class NestedData
{
    public required string LocationName { get; set; }
    public required double Latitude { get; set; }
    public required double Longitude { get; set; }
}
```

[Example 15-11](#) creates an instance of this model and then uses the `JsonConvert` class's `Serialize` method to serialize it to a string. It also

passes a second, optional, argument, a `JsonSerializationOptions`. I've used it here to indent the JSON to make it easier to read. (By default, `JsonSerializer` will use a more efficient layout with no unnecessary whitespace, but that is much harder to read.)

TIP

[Example 15-11](#) puts the `JsonSerializationOptions` in a static field because it is inefficient to construct new options each time you deserialize. (A code analyzer built into the .NET SDK will warn you if you do that.) In applications that process JSON repeatedly, `System.Text.Json` caches information about the types being serialized to avoid repeating expensive work. It stores that inside the `JsonSerializationOptions` object, which is why you should construct the settings just once and reuse them.

Example 15-11. Serializing data with `JsonSerializer`

```
private static readonly JsonSerializerOptions SerializeIndented =
    new() { WriteIndented = true };
public static string SerializeJson()
{
    var model = new SimpleData
    {
        Id = 42,
        Names = ["Bell", "Stacey", "her", "Jane"],
        Location = new NestedData
        {
            LocationName = "London",
            Latitude = 51.503209,
            Longitude = -0.119145
        },
        Map = new Dictionary<string, int>
        {
            { "Answer", 42 },
            { "FirstPrime", 2 }
        }
    };

    return JsonSerializer.Serialize(model, SerializeIndented);
}
```

The results look like this:

```
{
  "Id": 42,
  "Names": [
    "Bell",
    "Stacey",
    "her",
    "Jane"
  ],
  "Location": {
    "LocationName": "London",
    "Latitude": 51.503209,
    "Longitude": -0.119145
  },
  "Map": {
    "Answer": 42,
    "FirstPrime": 2
  }
}
```

As you can see, each .NET object has become a JSON object, where the name/value pairs correspond to properties in my model. Numbers and strings are represented exactly as you would expect. The `ICollection<string>` has become a JSON array, and the `IDictionary<string, int>` has become another JSON dictionary. I've used interfaces for these collections, but you can also use the concrete `List<T>` and `Dictionary<TKey, TValue>` types. You can use ordinary arrays to represent lists if you prefer. I tend to prefer the interfaces because it leaves you free to use whatever collection types you want. (E.g., [Example 15-11](#) initialized the `Names` property with a string array, but it could also have used `List<string>` without changing the model type.)

Converting serialized JSON back into the model is equally straightforward, as [Example 15-12](#) shows.

Example 15-12. Deserializing data with `JsonSerializer`

```
var deserialized = JsonSerializer.Deserialize<SimpleData>(json);
```

Although a plain and simple model such as this will often suffice, sometimes you may need to take control over some aspects of serialization, particularly if you are working with an externally defined JSON format.

For example, you might need to work with a JSON API that uses naming conventions that are different from .NET's—camelCasing is popular but conflicts with the PascalCasing convention for .NET properties. One way to resolve this is to use the `JsonPropertyName` attribute to specify the name to use in the JSON, as [Example 15-13](#) shows.

Example 15-13. Controlling the JSON with `JsonPropertyName` attributes

```
public class NestedData
{
    [JsonPropertyName("locationName")]
    public required string LocationName { get; set; }

    [JsonPropertyName("latitude")]
    public required double Latitude { get; set; }

    [JsonPropertyName("longitude")]
    public required double Longitude { get; set; }
}
```

`JsonSerializer` will use the names specified in `JsonPropertyName` when serializing and will look for those names when deserializing. This approach gives us complete control over the .NET and JSON property names, but there is a simpler solution for this particular scenario. This kind of renaming that just changes the case of the first letter is so common that you can get `JsonSerializer` to do it for you. [Example 15-14](#) constructs its `JsonSerializationOptions` by passing an optional argument of type `JsonSerializerDefaults`. By passing `JsonSerializerDefaults.Web`, we get the camelCasing style without needing to use any attributes. You can achieve the same effect by setting the `JsonSerializerOptions.PropertyNamingPolicy` to `JsonNamingPolicy.CamelCase`. .NET 8.0 adds four new naming styles: `KebabCaseLower` (looks-like-this), `KebabCaseUpper` (LOOKS-LIKE-THIS), `SnakeCaseLower` (looks_like_this), and `SnakeCaseUpper` (LOOKS_LIKE_THIS).

Example 15-14. Using JsonSerializerDefaults to get camelCased property names

```
private static readonly JsonSerializerOptions camelCaseOptions =
    new(JsonSerializerDefaults.Web) { WriteIndented = true };
public static string AutoCamelCase(SimpleData model)
{
    return JsonSerializer.Serialize(model, camelCaseOptions);
}
```

The `JsonSerializerOptions` also provide a way to handle circular references. Suppose you want to serialize objects of type `SelfRef`, as shown in [Example 15-15](#).

Example 15-15. A type supporting circular references

```
public class SelfRef
{
    public required string Name { get; set; }
    public SelfRef? Next { get; set; }
}
```

By default, if you attempt to serialize objects that refer to one another either directly or indirectly, you'll get a `JsonException` reporting a possible cycle. It says “possible” because it doesn't directly detect cycles by default—instead, `JsonSerializer` has a limit on the depth of any object graph that it will serialize. This is configurable through the `JsonSerializerOptions.MaxDepth` property, but by default the serializer will report an error if it has to go more than 64 objects deep. However, you can set the `ReferenceHandler` to change the behavior. [Example 15-16](#) sets this to `ReferenceHandler.Preserve`, enabling it to serialize a pair of `SelfRef` instances that refer to each other.

Example 15-16. Serializing a type supporting circular references

```
private static readonly JsonSerializerOptions jsonWithRefs =
    new(JsonSerializerDefaults.Web)
    {
        WriteIndented = true,
        ReferenceHandler = ReferenceHandler.Preserve
    };
```

```

public static string SerializeWithCircularReferences()
{
    var circle = new SelfRef
    {
        Name = "Top",
        Next = new SelfRef
        {
            Name = "Bottom",
        }
    };
    circle.Next.Next = circle;
    string json = JsonSerializer.Serialize(circle, jsonWithRefs);

    return json;
}

```

To enable this, the `JsonSerializer` gives objects identifiers by adding an `$id` property:

```

{
  "$id": "1",
  "name": "Top",
  "next": {
    "$id": "2",
    "name": "Bottom",
    "next": {
      "$ref": "1"
    }
  }
}

```

This enables the serializer to avoid problems when it encounters a circular reference. Whenever it has to serialize a property, it checks to see whether that refers to some object that has already been written out (or is in the process of being written out). If it does, then instead of attempting to write out the object again (which in this example would cause an infinite loop, since it'll just encounter the circular reference again and again), the serializer emits a JSON object containing a property with the special name `$ref` referring back to the relevant `$id`. This is not a universally supported form of JSON, which is why ID generation is not enabled by default.

You can control many other aspects of serialization with `JsonSerializerOptions`—you can define custom serialization mechanisms for data types, for example. (E.g., you might want to represent something as a `DateTimeOffset` in your C# code but have that become a string with a particular date-time format in the JSON.) The full details can be found in the `System.Text.Json` documentation.

In the examples shown so far, `JsonSerializer` discovers our types' properties using the reflection API. This is convenient, but there are a couple of disadvantages to this. The first time a process uses reflection is relatively slow, so if your code runs in an environment where *cold start* performance (i.e., the time it takes to be able to do useful work after the process has just started) is significant, reflection can be a problem. Some cloud hosting environments don't leave code running for very long on any single machine, so you can find that even a fairly busy service experiences a surprisingly large number of cold starts. Another problem with reflection is that it makes it harder for ahead-of-time compilation to be effective, because reflection-based serialization defers work until runtime—it can make it particularly difficult for trimming to remove unnecessary code from your build output, and might not work at all with Native AOT. For these reasons, the .NET SDK includes a code generator that can generate all of the type information required for serialization at compile time, removing any need to use reflection at runtime. [Example 15-17](#) shows how to enable this code generation.

Example 15-17. Enabling JSON serializer code generation

```
[JsonSerializable(typeof(SimpleData))]  
internal partial class CodeGenSerializationContext : JsonSerializerContext  
{  
}
```

Although this class appears to be empty, the `partial` keyword enables the code generator to supply an implementation. The generator looks for classes such as this that inherit from `JsonSerializerContext`, and which have at least one `JsonSerializable` attribute. It will emit code for each attribute, including a public property with a name matching the specified type. [Example 15-18](#) shows how to use the property generated for `SimpleData`.

Example 15-18. Using JSON serializer code generation

```
SimpleData? data = JsonSerializer.Deserialize(  
    json, CodeGenSerializationContext.Default.SimpleData);
```

For each `JsonSerializable` attribute, there will be a corresponding generated property on our serialization context class. When we pass this to `JsonSerializer`, it will use the type information that the code generator created, avoiding use of reflection, which can enable work to commence significantly more quickly the first time this code runs, and also making it possible to use Native AOT.

JSON DOM

Whereas `JsonSerializer` requires you to define one or more types representing the structure of the JSON you want to work with, `System.Text.Json` provides a fixed set of types that enable a more dynamic approach, and which can also support significantly more efficient JSON processing. You can build a Document Object Model (DOM) in which instances of types such as `JsonElement` or `JsonNode` represent the structure of the JSON.

`System.Text.Json` provides two ways to build a DOM. If you have data already in JSON form, you can use the `JsonDocument` class to obtain a read-only model of the JSON, in which each object, value, and array is represented as a `JsonElement`, and each property in an object is represented as a `JsonProperty`. [Example 15-19](#) uses `JsonDocument` to discover all of the properties in the object at the root of the JSON by calling `RootElement.EnumerateObject()` on the `JsonDocument`. This returns a collection of `JsonProperty` structs.

Example 15-19. Dynamic JSON inspection with `JsonDocument` and `JsonElement`

```
using (JsonDocument document = JsonDocument.Parse(json))  
{  
    foreach (JsonProperty property in document.RootElement.EnumerateObject())  
    {  
        Console.WriteLine($"Property: {property.Name} ({property.Value.ValueKind})")  
    }  
}
```

```
}  
}
```

Running this on the serialized document produced by earlier examples produces this output:

```
Property: id (Number)  
Property: names (Array)  
Property: location (Object)  
Property: map (Object)
```

As this shows, we are able to discover at runtime what properties exist. The `JsonProperty.Value` returns a `JsonElement` struct, and we can inspect its `ValueKind` to discover which sort of JSON value it is. If it's an array, we can enumerate its contents by calling `EnumerateArray`, and if it's a string value, we can read its value by calling `GetString`. [Example 15-20](#) uses these methods to show all the strings in the `names` property.

Example 15-20. Dynamic JSON array enumeration with `JsonDocument` and `JsonElement`

```
JsonElement namesElement = document.RootElement.GetProperty("names");  
foreach (JsonElement name in namesElement.EnumerateArray())  
{  
    Console.WriteLine($"Name: {name.GetString()}");  
}
```

As this example also shows, if you know in advance that a particular property will be present, you don't need to use `EnumerateObject` to find it: you can call `GetProperty`. There's also a `TryGetProperty` for when the property is optional. [Example 15-21](#) uses both: this treats the root object's `location` property as optional, but if it is present, it then requires the `locationName`, `latitude`, and `longitude` properties to be present.

Example 15-21. Reading JSON properties with `JsonElement`

```
if (root.TryGetProperty("location", out JsonElement locationElement))  
{  
    JsonElement nameElement = locationElement.GetProperty("locationName");  
    JsonElement latitudeElement = locationElement.GetProperty("latitude");
```

```

        JsonElement longitudeElement = locationElement.GetProperty("longitude");
        string locationName = nameElement.GetString();
        double latitude = latitudeElement.GetDouble();
        double longitude = longitudeElement.GetDouble();
        Console.WriteLine($"Location: {locationName}: {latitude},{longitude}");
    }

```

In addition to structural elements, objects and arrays, the data model in the [JSON specification](#) recognizes four basic data types: strings, numbers, Booleans, and null. As you've seen, you can discover which of these any particular `JsonElement` represents with its `Kind` property. If it's one of the basic data types, you can use a suitable `Get` method. The last two examples both used `GetString`, and the second also used `GetDouble`.

There are multiple methods you can use to retrieve a number: if you are expecting an integer, you can call `GetSByte`, `GetInt16`, `GetInt32`, or `GetInt64` (and unsigned versions are also available) depending on what range of values you are expecting. There's also `GetDecimal`.

`JsonElement` also offers methods for reading string properties in particular formats: `GetGuid`, `GetDateTime`, `GetDateTimeOffset`, and `GetBytesFromBase64`.

All of the `Get` methods will throw an exception if the value is not in the required form, but it won't always be the same exception. If the JSON has a type that can't possibly contain a suitable value (e.g., you call `GetGuid` when the property's value is `false`, or you call `GetInt32` when the property is a string) it will throw `InvalidOperationException`. If the JSON type is appropriate but the value is not right (e.g., you called `GetUInt32` but the number is negative, or is a floating-point value, or you called `GetDateTime` and the JSON value is a string, but is not a valid ISO 8601 datetime) it will throw a `FormatException` instead.

Each of these `Get` methods is also available in a `TryGet` form, such as `TryGetUInt32`. These enable you to detect when the data cannot be parsed in the expected way without triggering an exception, although there's a trap for the unwary: the `TryGet` forms will throw an `InvalidOperationException` in the same cases where the equivalent `Get` would—these only return `false` in cases where the equivalent `Get` method would have thrown a `FormatException`. If you want to avoid exceptions entirely, you should check the element's `ValueKind` property, and only call the `TryGet` method if that indicates that the JSON type is what you expect.

These types attempt to minimize the amount of memory allocated.

`JsonElement` and `JsonProperty` are both structs, so you can obtain these without causing additional heap allocations. The underlying data is held in UTF-8 format by the `JsonDocument`, and the `JsonElement` and `JsonProperty` instances just refer back to that, avoiding the need to allocate copies of the relevant data. This can make these types significantly more efficient than deserialization in some scenarios.

Obviously, the underlying data does need to live somewhere, and depending on exactly how you loaded the JSON into a `JsonDocument`, it may have to allocate some memory to hold it. (E.g., you can pass it a `Stream`, and since not all streams are rewindable, `JsonDocument` would need to make a copy of the stream's contents.) `JsonDocument` uses the buffer pooling features available in the .NET runtime libraries to manage this data, meaning that if an application parses many JSON documents, it may be able to reuse memory, reducing pressure on the garbage collector (GC). But this means the `JsonDocument` needs to know when you've finished with the JSON so that it can return buffers to the pool. That's why we use a `using` statement when working with a `JsonDocument`.

WARNING

Be aware that `JsonElement.GetString` is more expensive than all the other `Get` methods, because it has to create a new .NET string on the heap. The other `Get` methods all return value types, so they do not cause heap allocations.

I mentioned earlier that there are two ways of working with a JSON DOM. `JsonDocument` provides a read-only model that lets you inspect existing JSON. But there is also `JsonNode`, which is read/write. You can use this in a couple of ways that `JsonDocument` does not support. You can build up an object model from scratch to create a new JSON document. Alternatively, you can parse existing JSON into an object model just like with `JsonDocument`, but when you use `JsonNode`, the resulting model is modifiable. So you could use it to load some JSON and modify it, as [Example 15-22](#) illustrates.

Example 15-22. Modifying JSON with `JsonNode`

```
JsonNode rootNode = JsonNode.Parse(json)!;  
JsonNode mapNode = rootNode["map"]!;  
mapNode["iceCream"] = 99;
```

This loads the JSON text in `json` into a `JsonNode` and then retrieves the `map` property. (This example expects to work with JSON in the same form as I've used in the preceding examples, with camelCased property names.) So far this doesn't do anything we couldn't do with `JsonDocument`. But the final line adds a new entry to the object in `map`. It's this ability to modify the document that makes `JsonNode` more powerful. So why do we need `JsonDocument` if `JsonNode` is more powerful? The power comes at a price: `JsonNode` is less efficient, so if you don't need the extra flexibility, you shouldn't use it.

An advantage of using either the read-only `JsonDocument` and `JsonElement` or the writable `JsonNode` is that you don't need to define any types to model the data. They also make it easier to write code whose behavior is driven by the structure of the data, because these APIs are able to describe what they find. The read-only form is typically more efficient than `JsonSerializer`, because it may enable you to cause fewer object allocations when reading data from a JSON document.

Summary

The `Stream` class is an abstraction representing data as a sequence of bytes. A stream can support reading, writing, or both, and may support seeking to arbitrary offsets as well as straightforward sequential access. `TextReader` and `TextWriter` provide strictly sequential reading and writing of character data, abstracting away the character encoding. These types may sit on top of a file, a network connection, or memory, or you could implement your own versions of these abstract classes. The `FileStream` class also provides some other filesystem access features, but for full control, we also have the `File` and `Directory` classes. When bytes and strings aren't enough, .NET offers various serialization mechanisms that can automate the mapping between an object's state in memory and a representation that can be written out to disk or sent over the

network or any other stream-like target; this representation can later be turned back into an object of the same type and with equivalent state.

As you've seen, a few of the file and stream APIs offer asynchronous forms that can help improve performance, particularly in highly concurrent systems. The next chapter tackles concurrency, parallelism, and the task-based pattern that the asynchronous forms of these APIs use.

- 1 You might have thought that the pound sign was #, but if, like me, you're British, that's just not on. It would be like someone insisting on referring to @ as a dollar sign. Unicode's canonical name for # is *number sign*, and it also allows my preferred option, *hash*, as well as *octothorpe*, *crosshatch*, and, regrettably, *pound sign*.
- 2 Just in case you've not come across the term, in *little-endian* representations, multibyte values start with the lower-order bytes, so the value 0x1234 in 16-bit little-endian would be 0x34, 0x12, whereas the big-endian version would be 0x12, 0x34. Little-endian looks reversed, but it's the native format for Intel's processors.
- 3 Some Unicode characters can take up to 4 bytes in UTF-8, so multiplying by three might seem like it could underestimate. However, all such characters require two code units in UTF-16. Any single `char` in .NET will never require more than 3 bytes in UTF-8.
- 4 Although `FileStream` normally opens the file, there are constructors that accept a file handle.
- 5 `FileMode` only comes into play at the instant when a file is opened, so you don't need it with the constructors that accept a file handle.
- 6 These all return a `DateTime` that is relative to the computer's current time zone. Each of these methods has an equivalent that returns the time relative to time zone zero (e.g., `GetCreationTimeUtc`).
- 7 To use this on .NET Framework, you will need to add a reference to the `System.Text.Json` NuGet package.